GHULAM ISHAQ KHAN INSITITUTE OF ENIGNEERING SCIENCES AND TECHNOLOGY - GIKI

THEORY OF COMPUTATION AND AUTOMATA – CS224

FACULTY OF COMPUTER SCIENCES AND ELECTRICAL ENGINEERING

“CONTEXT FREE GRAMMAR AND PUSH DOWN AUTOMATA”

DETAIL NOTES

BY: MUHAMMAD SAJID ALI

TABLE OF CONTENTS:

INTRODUCTION TO CONTEXT FREE GRAMMARS.....PgNo.1

PARSE TREES.....PgNo.2

AMBIGOUS GRAMMARS.....PgNo.2

SIMPLIFICATION OF CFGS.....PgNo.3

CHOMSKY NORMAL FORM.....PgNo.4

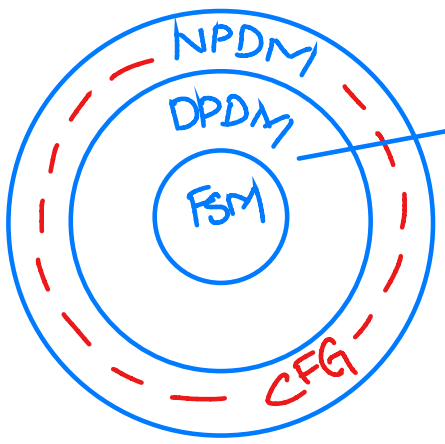
PUSH DOWN AUTOMATA.....PgNo.5

CFG to PDA.....PgNo.7

CONTEXT FREE GRAMMARS — CFGs.

So far we have defined languages using.
(i) Regular Expressions.
(ii) Finite Automata

CFG is a method of defining languages:
→ With more powerful Machines.
→ However, it still may not define all possible languages.
→ CFGs are useful for defining nested structures.
e.g: paranthesis in programming.
→ CFGs is a recursive R.E.



→ Describes LR(K) Grammar; compiler's build; most of all the programming languages are defined by LR(K).

CONCEPT

All the language cannot be defined by R.E and FA.
e.g:- Regex for balanced parathensis?

{}, {{}}, {{{}}}

R.E ⇒ {} | {{}} | {{{}}} ... but like this we would have to make infinitely long regular expression to capture every possible pattern.

Try using {*}* as a RE for our required problem. do you see any problem?

The solution to our problem is through CFGs.
We need to allow our regular expressions to refer back to themselves.

S → {S} | ∧ ⇒ this is a CFG.

DEFINITIONS

① An alphabet of letters called terminals. (will be the words of the language).

② Non-terminals are phrases or type of symbols like S indicating to start from here.

③ Set of Productions forms rules of our CFG.

Note: one nonterminal —goes to→ finite string (check ① for reference) of terminals and/or non terminals.

Example: S → S | 0 | 0S1 —— ①
S → ∧ —— ②
S: Non terminal; 0, 1, ∧: Terminal; ①, ②: set of production (s)

→ A language generated by a CFG is the set of all strings of terminals that can produce from start symbol 'S' using production Rules.

→ A language generated by a CFG is called a 'context free language; 'CFL'.

Example no. 1:

S → 0S1 | e

S → 0S1 → S → 00S11
→ 0011.

This given CFG can produce 0^n1^n with equal no.s of zeros and 1s, including empty string.

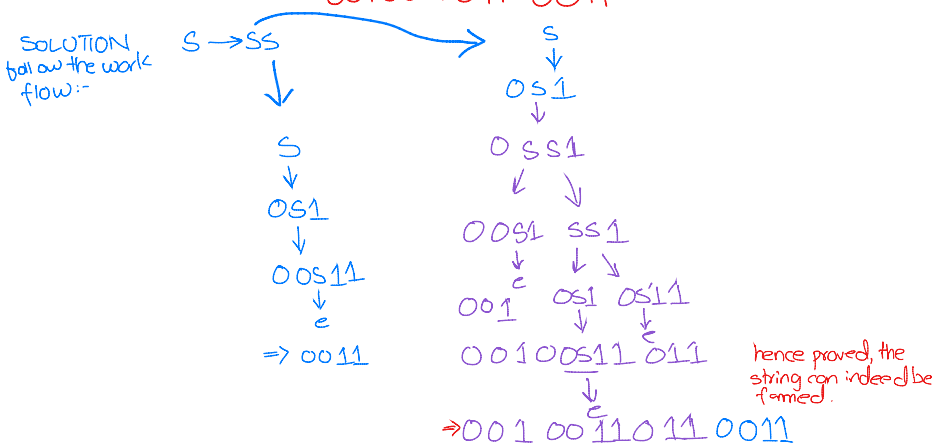
Example no. 2

S → 0S1 | SS | e.

Form the language produced by this CFG —?

Does the defined CFG produces the string 00100110110011

SOLUTION follow the work flow:-



hence proved, the string can indeed be formed.

Note: In the previous example after moving from $S \rightarrow SS$, we had two choices.

$S \rightarrow SS$ → choice of selecting which non-terminal for derivation of required output first.

leftmost derivation. rightmost derivation.

DERIVATION: Sequence of string that typically have both terminals and non terminals.

Example no. 3:

Construct CFG for $\{0^n 1^n \mid n \geq 1\}$

Solution:

$S \rightarrow 0S1$.
$S \rightarrow 01$

here, $\{0,1\} \in$ Terminals

$\{S\} \in$ Variables non terminals.

Example no. 4. $S \Rightarrow$ start symbol

Grammar for unsigned integers is given as

$U \rightarrow UD \mid D$

$D \rightarrow 0|1|2|3|4|5|6|7|8|9$

Generate: 1001 from this grammar —?

Ambiguous Grammar

A grammar is said to be ambiguous if there exists (for a certain string) more than one derivations.

i.e 2 or more 'left/Right' derivation trees.

Example no. 4.

Consider the string 'a+a*b' generated from $S \rightarrow S+S \mid S*S \mid a \mid b$

1st Method of derivation

$S \rightarrow S+S$
$S \rightarrow a+S$
$S \rightarrow a+S*S$
$S \rightarrow a+a*S$
$S \rightarrow a+a*b$

2nd Method of derivation.

$S \rightarrow S*S$
$S \rightarrow S+S*S$
$S \rightarrow a+S*S$
$S \rightarrow a+a*S$
$S \rightarrow a+a*b$.

Note how for a single string 2 left most derivations exist.

∴ Ambiguous grammar.

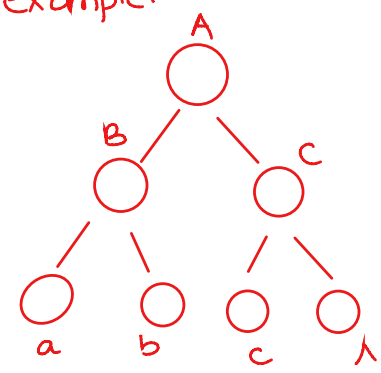
Parse Trees

A graph that shows a particular string, how it was derived using some 'CFG'.

→ it has a direct relation with with left and right most derivations.

→ Expresses as λ an important concept in CFG.

Sample example:



terminals/
(i) λ a, b, c, $\wedge \in$ Leave nodes

(ii) A, B, C $\in$ interior nodes/ non-terminals

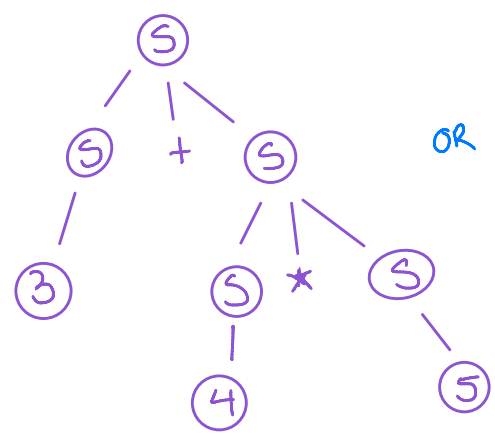
Root Node: Must start from the start symbol

Example no. 5:-

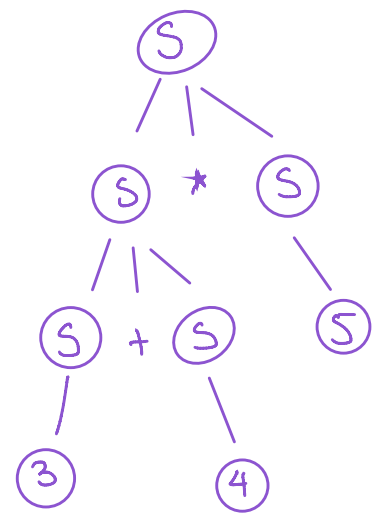
Consider the example of Parse tree from the CFG given at Example. 4:-

Consider:- $S \rightarrow S+S \mid S*S \mid a \mid b$

(i) 3+4*5



OR



Generate parse trees for —?

((3+4)*5)

(3+(4*5))

Example no. 6: Construct a CFG for a^*.

$S \rightarrow SS$
$S \rightarrow a$
$S \rightarrow \wedge$

Let us generate aa to verify:

$S \rightarrow SS$
$S \rightarrow SSS$
$S \rightarrow SSSS$
$S \rightarrow \wedge SSS$
$S \rightarrow \wedge aSS$

$S \rightarrow \wedge aaS$
$S \rightarrow \wedge aa\wedge$

Is the given grammar Ambigious?

* page no. 235 (Home Work)

Example no 7: $S \to aSb$
$S \to \wedge$

Is the given CFG ambigious?

Example no 8: Construct a CFG for palindrome.

$S \to aSa$
$S \to bSa$
$S \to \wedge$

let us verify the CFG by constructing abaaba.

$S \to aSa$
$S \to abSba$
$S \to abaSaba$
$S \to aba\wedge aba$
$S \to abaaba$.

Example no 8: Construct a CFG for $(a+b)^* aa (a+b)^*$

$S \to XaaX$
$X \to aX|bX|\wedge$.

Simplification of CFGs.

following are the steps per formed for the Simplification of a CFG.

① Removal of NULL /∧ Productions.

Procedure

step no. 1: find out all the productions on right side that contain ∧.
(Non-terminal can be replaced with ∧).

step no. 2: Replace that Non-terminal with ∧ for each occurance.

step no. 3: Add the resultant productionto the Grammar.

Example no. 9:

$S \to aX$
$X \to \wedge$

step no. 1: $X \to \wedge$

$S \to aX$
$S \to a$

Example no. 10:

$S \to a | Xb | aYa$
$X \to Y | \wedge$
$Y \to b | X$

Step no. 1: $S \to Xb$
$Y \to X$

$S \to b$
$Y \to \wedge$
$\Rightarrow S \to aYa$
$S \to aa$

$\therefore S \to a|b|Xb|aYa|aa$
$X \to Y$
$Y \to b$

Example no. 11:

$S \to asa | bsb | \wedge$

step1 $S \to \wedge$
$S \to asa$
$S \to aa$

step no. 2
$S \to bSb$
$S \to bb$

$\therefore S \to asa | bSb | aa | bb$ Ans!

Example no. 12

$S \to ABAC$, $A \to aA|\wedge$
$B \to bB|\wedge$
$C \to c$

Remove NULL productions from the given CFG.

② Removal of Unit Productions.

@ — $A \to B$ where if A, B ∈ nonterminal then @ is a unit production.

Procedure:

Step no.1 Add production;
$A \to X$ to the grammar rule
when-ever $B \to X$ occurs.

Step no.2 [$X \in$ Terminal / Λ]
Delete $A \to B$ from the grammar

Step no.3
Repeat steps for all unit productions (until all are removed).

Example no.13:

$S \to A \mid bb$
$A \to B \mid b$
$B \to S \mid a$.

Unit productions:-
(i) $S \to A$
(ii) $A \to B$
(iii) $B \to S$

$S \to A$, gives $S \to b$
$S \to A \to B$, gives $S \to a$

$\therefore$ $S \to b \mid a \mid bb$
$A \to B \mid b$
$B \to S \mid a$

$A \to B$

$A \to B$ gives $A \to a$

$\therefore$ $S \to b \mid a \mid bb$
$A \to a \mid b$
$B \to S \mid a$.

$B \to S$

$B \to S$ giv $B \to b$
$\parallel$ $B \to a$
$\parallel$ $B \to bb$

$S \to b \mid a \mid bb$
$A \to a \mid b$
$B \to b \mid a \mid bb$

Example no. 14

Classpractice

$S \to XY$
$X \to a$
$Y \to Z \mid b$
$Z \to M$
$M \to N$
$N \to a$

Remove NULL productions from the given CFG.

CHOMSKY - Normal Form

A special form of CFG that has a length restriction, allowing the CFG at RHS to only contain.

(i) 2 Terminals
OR
(ii) 1 NonTerminal / Variable.

All productions should be of the following form.

Nonterminal $\to$ Two - Nonterminal(s)
OR
Terminal $\to$ One terminal.

Example no.14.

① ② ③ ④ ⑤ ⑥
$S \to aSa \mid bSb \mid a \mid b \mid aa \mid bb$

step no.1: No Λ / Lambda productions.

step no.2:- No unit productions.
(if new unit productions created later in the process, remove it.)

Step no.3:- For every production involving terminals, replace each with non-terminals

① $S \to aSa \Rightarrow ASA$.
and add new production to the grammar $A \to a$.
Repeat this step for all the similar productions.

② $S \to bSb \Rightarrow S \to BSB$
$B \to b$

$S \to aa \Rightarrow S \to AA$.

⑥ $S \to bb \Rightarrow S \to BB$

③ & ④ are already in CNF

① $S \to ASA$ ② $S \to BSB$ ③ $S \to AA$.
④ $S \to BB$ ⑤ $S \to a$ ⑥ $S \to b$
⑦ $A \to a$ ⑧ $B \to b$.

step no. 4 :- for every production of the following form:

Nonterminal → string of Non terminals > 2.

Reduce them to 2 non terminals by introducing a new nonterminal (e.g: R1).

① $S \to ASA$
$S \to AR_1$
$R_1 \to SA$

② $S \to BSB$
$S \to BR_2$
$R_2 \to SB$

and the process shall continue.... (for all).

Resulting GRAMMAR!

$S \to AR_1$
$R_1 \to SA$
$S \to BR_2$
$R_2 \to SB$

$S \to AA$
$S \to BB$
$S \to a$
$S \to b$

$A \to a$
$B \to b$.

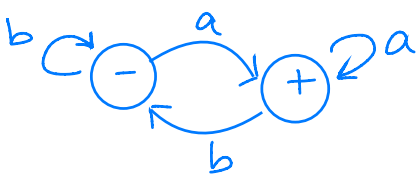
Solve another question for practice.

Q. $S \to bA \mid aB$
$A \to bAA \mid aS \mid a$
$B \to aBB \mid bS \mid b$.

PUSHDOWN AUTOMATA

Background.

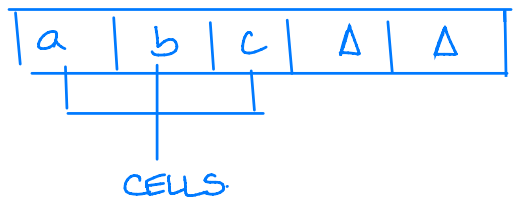
Consider the FA. (i) Noway, we describe when input is ended



(ii) from where we read the input string...

Consider another way. means that the input string ends here.

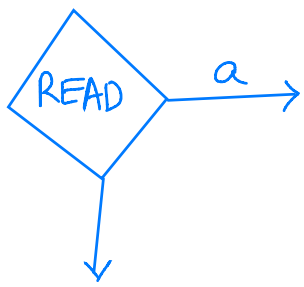
Input tape



START ..... +

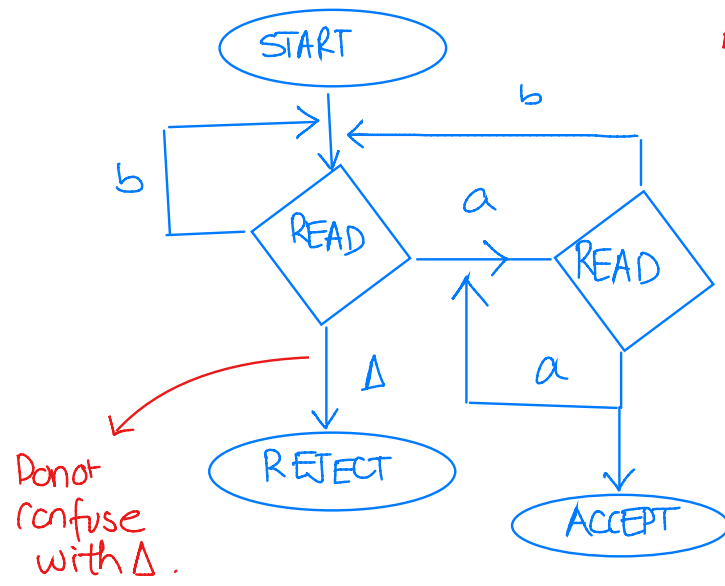
ACCEPT ..... −

REJECT ..... ○ dead end!



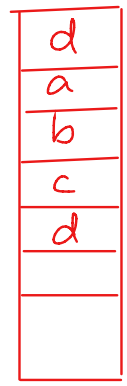
..... Decisions / Transitions.

Example no. 15.



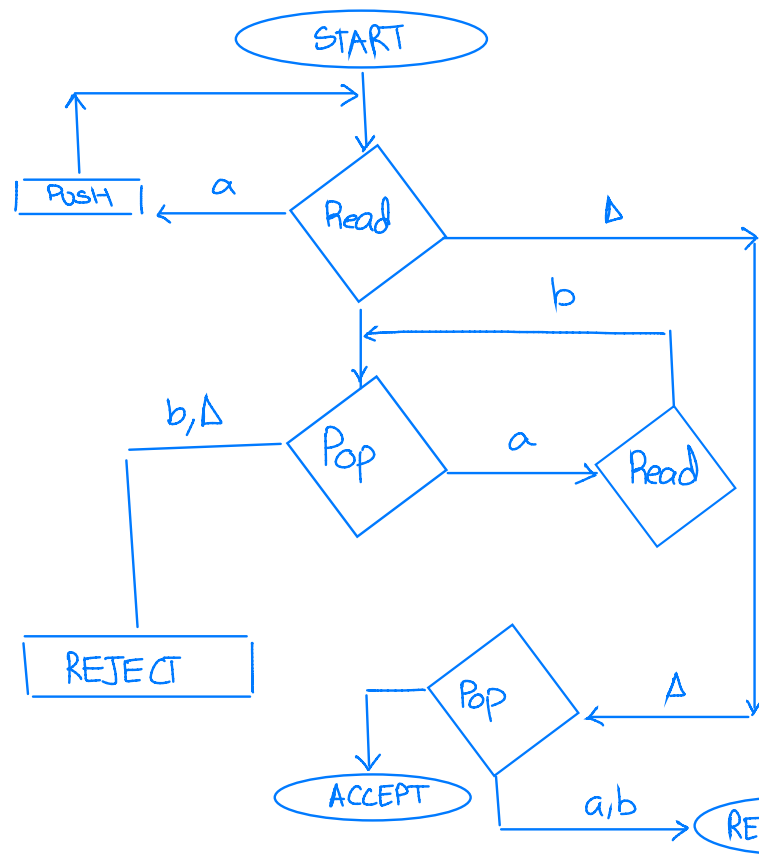
Donot confuse with Δ.

Adding a PUSH-DOWN Stack



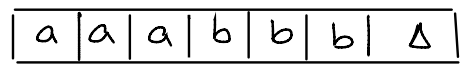
Stack operations → initially empty
(i) PUSH.
(ii) POP.
→ Follows LIFO Mechanism.

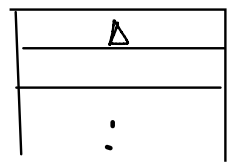
Example no. 16



See if this PDA accepts the string 'aaabbb'.

Tape



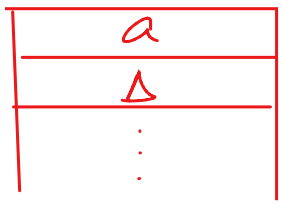


STACK.

following procedure!

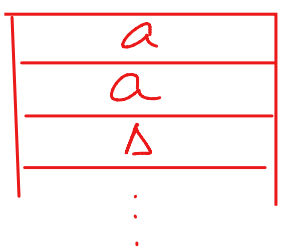
PUSH.

A	a	a	b	b	b	Δ



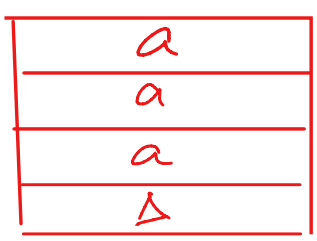
PUSH

A	A	a	b	b	b	Δ



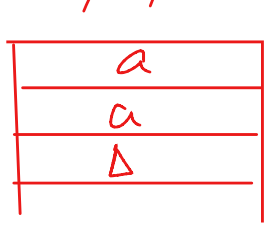
PUSH

A	A	A	b	b	b	Δ



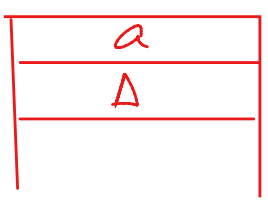
Pop

A	A	A	b	b	b	Δ



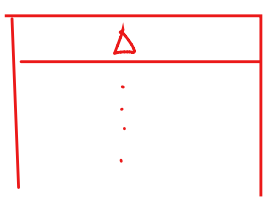
Pop

A	A	A	b	b	b	Δ



Pop

A	A	A	b	b	b	Δ



The given PDA thus, does accepts the strings of $a^n b^n$.

Note: (i) Such a language (i.e $a^n b^n$) cannot be accepted by the finite automata; because it cannot keep track that how many times it looped for a^n.

(ii) The stack has unlimited capacity.

FORMAL DEFINITION:-

PUSH DOWN automata is away to implement a CFG in a similar way; as we designed FA for a regular grammar.

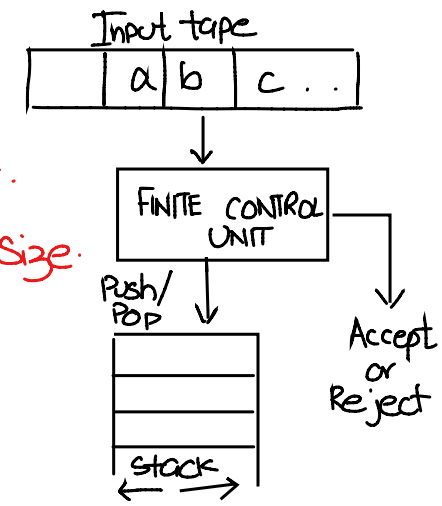
(i) It is more powerful than FSM.

(ii) FSM has very limited memory but PDA has more memory.

In general :- PDA = FSM + stack!

COMPONENTS OF PDA:-

(i) Input tape
(ii) Finite Control Unit.
(iii) Stack with finite size.



Further Defining.

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

Q = finite set of states.

Σ = Finite set of Input symbols.

Γ = A finite stack of Alphabets

δ = The transition function.

q_0 = Start state.

Z_0 = the start state symbol.

F = the set of final / accepting states

Note:

As Γ represents the finite set of alphabets for stack. It means we do not need to use the same alphabet as input string.

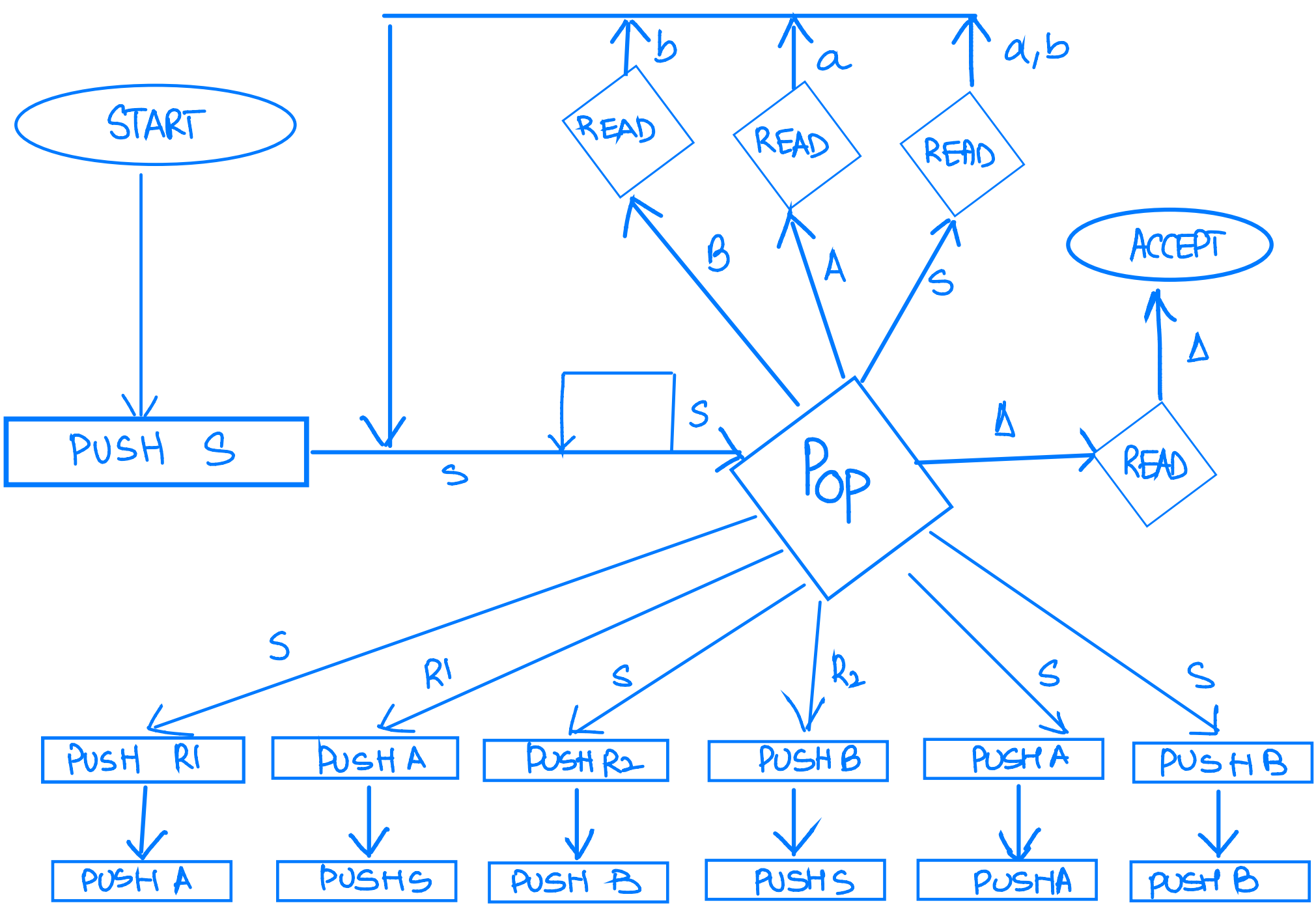
Example no. 17

Convert the give CFG to PDA.

$S \rightarrow AR_1$
$R_1 \rightarrow SA$
$S \rightarrow BR_2$
$R_2 \rightarrow SB$
$S \rightarrow AA$
$S \rightarrow BB$

$S \rightarrow a$
$S \rightarrow b$
$A \rightarrow a$
$B \rightarrow b$
$S \rightarrow \Lambda$

SOLUTION



Analyze rules for CFG→PDA at page 234.

Question continued:

Examine abaaba through the resultant PDA and show tape, state and stack.

Deriving given string using the given CFG.

$S \Rightarrow AR_1$
$\Rightarrow aR_1$
$\Rightarrow aSA$
$\Rightarrow aBR_2A$
$\Rightarrow abR_2A$
$\Rightarrow abSBA$
$\Rightarrow abAABA$
$\Rightarrow abaABA$
$\Rightarrow abaaBA$
$\Rightarrow abaaba.$

LEFTMOST DERIVATION	STATE	TAPE	STACK
	START	abaaba	Δ
	PUSH S	abaaba	S
	POP	abaaba	Δ
	PUSH R1	abaaba	R1
$S \Rightarrow AR_1$	PUSH A	abaaba	AR1
	POP	abaaba	R1
$\Rightarrow aR_1$	READ a	abaaba	R1
	POP	abaaba	Δ
	PUSH A	abaaba	A
$\Rightarrow aBA$	PUSH S	abaaba	SA
	POP	abaaba	A
	PUSH R2	abaaba	R2A
$\Rightarrow aBR_2A$	PUSH B	abaaba	BR2A
	POP	abaaba	R2A
$\Rightarrow abR_2A$	READ b	abaaba	R2A
	POP	abaaba	A
	PUSH B	abaaba	BA
$\Rightarrow abSBA$	PUSH S	abaaba	SBA
	POP	abaaba	BA
	PUSH A	abaaba	ABA
$\Rightarrow abAABA$	PUSH A	abaaba	AABA
	POP	abaaba	ABA
$\Rightarrow abaABA$	Read a	abaaba	ABA
	POP	abaaba	BA
$\Rightarrow abaaBA$	Read a	abaaba	BA
	POP	abaaba	A
$\Rightarrow abaabA$	Read b	abaaba	A
	POP	abaaba	A
	Read a	abaaba	Δ
—	—	—	—